

Universidade Federal do Rio Grande do Norte
Instituto Metrópole Digital
PROGRAMAÇÃO DISTRIBUÍDA



Prof. Nelio Alessandro Azevedo Cacho

Discente
MAYCON DOUGLAS RÊGO SANTOS

Sumário

Sumário.....	1
1. Arquitetura da Aplicação.....	3
1.1 Visão Geral e Componentes.....	3
1.2 Responsabilidades dos Componentes.....	4
2. Implementação dos Componentes.....	4
2.1 Detalhes de Implementação.....	4
2.2 Padrões de Projeto GoF Utilizados.....	6
2.3 Padrões de Sistemas Distribuídos.....	7
3. Suporte a Múltiplos Protocolos.....	7
3.1 Protocolo UDP.....	7
3.2 Protocolo TCP.....	8
3.3 Protocolo HTTP.....	8
3.4 Protocolo gRPC.....	8
4. Ambiente de Testes na AWS.....	9
4.1 Instâncias AWS EC2 e Configurações.....	9
4.2 Endereços IP e Mapeamento de Rede.....	9
4.3 Outros Recursos Relevantes.....	10
5. Resultados dos Testes de Carga.....	11
5.1 Testes de Carga por Protocolo (UDP, TCP, HTTP).....	11
6. Discussão e Análise dos Resultados.....	13
6.1 Análise Comparativa dos Protocolos.....	13
6.2 Considerações e Conclusões.....	14

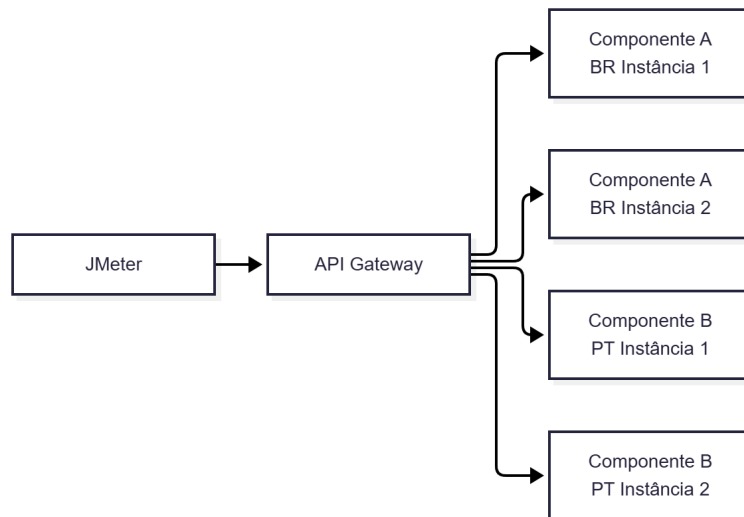
1. Arquitetura da Aplicação

1.1 Visão Geral e Componentes

A aplicação é um sistema distribuído mínimo cujo domínio é devolver o horário atual de duas zonas geográficas: Brasil (America/Sao_Paulo) e Portugal (Europe/Lisbon).

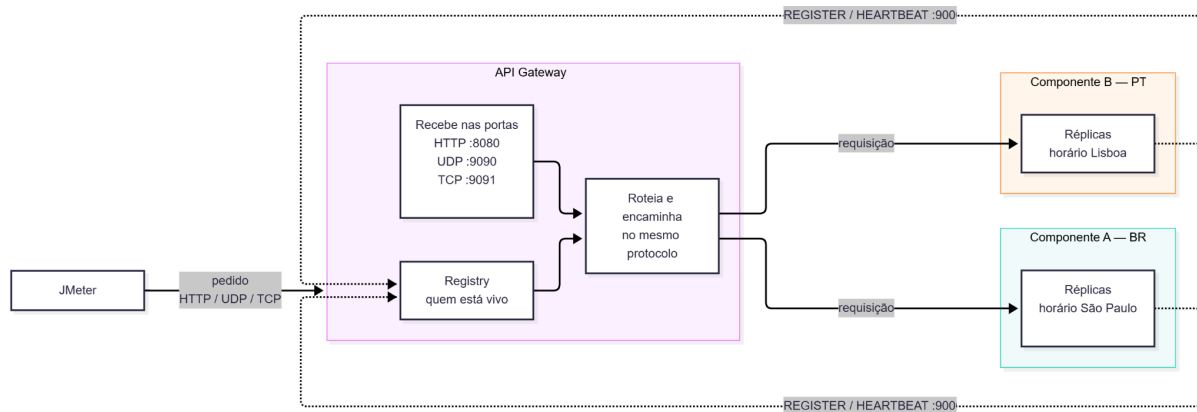
Em resumo, um API Gateway que recebe pedido de horário (BR ou PT) por HTTP, UDP ou TCP, descobre instâncias vivas por heartbeat, escolhe uma por round-robin, e encaminha no mesmo protocolo até a instância

O cliente (JMeter) fala só com o **API Gateway**. Cada componente (BR ou PT) pode ter várias réplicas. O gateway escolhe uma instância viva por round-robin



Papel	Função	Processo
JMeter	Gera as requisições	—
API Gateway	Entrada, registro e roteamento	GatewayMain
Componente A	Horário BR (stateless)	InstanceMain br ...
Componente B	Horário PT (stateless)	InstanceMain pt ...

1.2 Responsabilidades dos Componentes



Seta contínua = pedido do cliente. Seta tracejada = descoberta e monitoramento das instâncias.

- **JMeter:** envia carga só para o gateway (não fala direto com as instâncias).
- **API Gateway:** recebe o pedido nas portas acima, mantém quem está vivo (:9000), escolhe uma réplica e encaminha no mesmo protocolo da entrada.
- **Componente A (BR):** registra-se, manda heartbeat e devolve o horário de São Paulo.
- **Componente B (PT):** igual ao A, com horário de Lisboa.

2. Implementação dos Componentes

2.1 Detalhes de Implementação

O código do lab está em `br.imd.ufrn.lab`. Em execução há dois processos: o gateway (`GatewayMain`) e cada réplica BR/PT (`InstanceMain`).

Pacotes

`br.imd.ufrn`

```
|— Main.java      entrada (gateway | br | pt)
|— model/         InstanceInfo
|— gateway/       processo do Gateway
|   |— GatewayMain, HeartbeatServer, InstanceRegistry
|   |— ClientHttpServer, ClientUdpServer, ClientTcpServer
|   |— TimeRequestHandler, TransportProtocol
|   |— TcpForwarder, UdpForwarder, HttpForwarder
|— instance/      processo de cada instância
|   |— InstanceMain, HeartbeatClient
|   |— InstanceTcpServer (TIME em TCP + demux HTTP)
|   |— InstanceUdpServer (TIME em UDP, mesma porta)
```

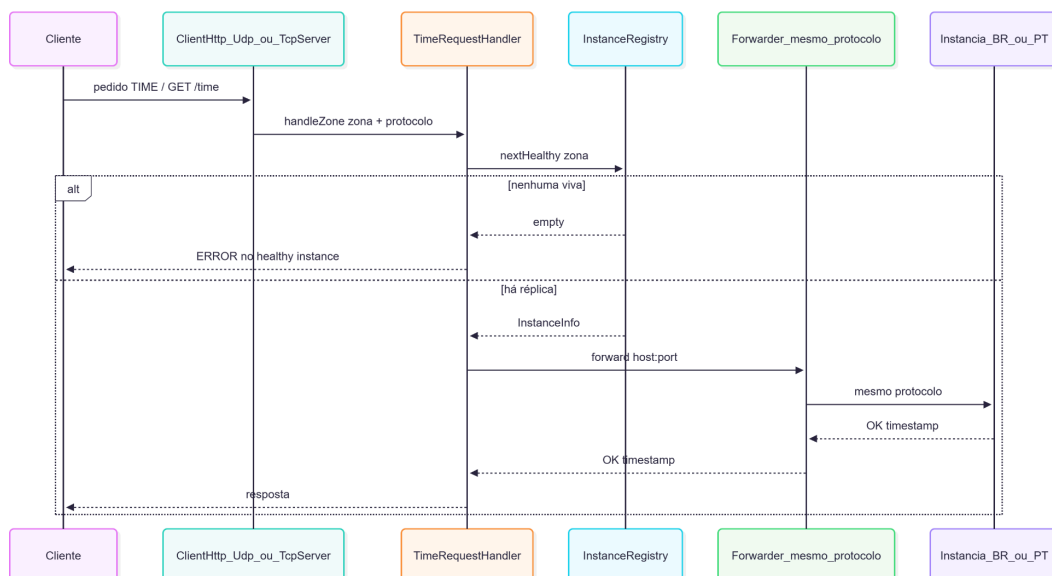
API Gateway

Ao iniciar, GatewayMain monta um InstanceRegistry, três forwarders (TcpForwarder, UdpForwarder, HttpForwarder) e um TimeRequestHandler. Em seguida sobe quatro threads (Runnable):

Thread	Classe	Porta	Função
Controle	HeartbeatServer	9000	REGISTER/HEARTBEAT/LIST
Cliente HTTP	ClientHttpServer	8080	GET /time/br, /time/pt, /registry
Cliente UDP	ClientUdpServer	9090	datagram TIME br pt
Cliente TCP	ClientTcpServer	9091	linha TIME br pt

Um agendador remove do registry as instâncias cujo último heartbeat passou de ~6s. O núcleo do roteamento é o TimeRequestHandler:

1. Valida a zona (br ou pt).
2. Chama registry.nextHealthy(zona) (round-robin só entre vivas).
3. Encaminha com o forwarder do mesmo TransportProtocol da entrada.
4. Devolve OK <timestamp> ou ERROR ... ao servidor de cliente.



InstanceRegistry guarda um ConcurrentHashMap<String, InstanceInfo> (chave = instanceId) e um contador AtomicInteger por tipo (br / pt) para o round-robin. InstanceInfo guarda tipo, host, porta e lastSeenMillis.

Componente A / B (instância)

Cada InstanceMain <br|pt> <id> <porta> representa uma réplica do componente A (BR) ou B (PT). No start:

1. HeartbeatClient envia REGISTER e, a cada ~3s, HEARTBEAT <id> no :9000 do gateway.
2. InstanceUdpServer escuta UDP na porta informada.
3. InstanceTcpServer escuta TCP na mesma porta e faz demux: se a primeira linha começa com GET , trata HTTP; senão, trata TIME br|pt.

A zona é fixa no processo: BR usa America/Sao_Paulo, PT usa Europe/Lisbon. Se o pedido pedir a zona “errada”, a instância responde ERROR...

Concorrência e resiliência

- Servidores de cliente e heartbeat usam virtual threads por conexão/pacote.
- Estado do registry é concurrent-safe (ConcurrentHashMap).
- Falha de forward vira ERROR forward failed (ou HTTP 502 no gateway).
- Matar uma réplica: após o timeout ela sai do round-robin; as restantes continuam atendendo. Subir de novo com o mesmo (ou novo) id e REGISTER a re insere.

2.2 Padrões de Projeto GoF Utilizados

Facade [TimeRequestHandler]

Os três servidores de cliente (ClientHttpServer, ClientUdpServer, ClientTcpServer) não conhecem registry nem sockets das instâncias. Eles só chamam handleZone / handleLine. O handler concentra validação, escolha da réplica e seleção do forwarder.

Strategy [encaminhamento por protocolo]

O enum TransportProtocol (TCP, UDP, HTTP) escolhe, em tempo de execução, qual “estratégia” de transporte usar: TcpForwarder, UdpForwarder ou HttpForwarder. O contrato é o mesmo (levar o pedido até InstanceInfo e trazer a resposta em texto), mudando só o mecanismo de rede.

Singleton [por processo]

Em cada JVM do gateway existe um único InstanceRegistry e um único conjunto de listeners criados em GatewayMain. Isso evita registries duplicados e inconsistência de quem está “vivo”.

2.3 Padrões de Sistemas Distribuídos

API Gateway

GatewayMain é o único ponto de entrada do cliente. JMeter e scripts falam com :8080 / :9090 / :9091, as réplicas A/B não precisam ser conhecidas a priori pelo cliente. O gateway também concentra a política de roteamento e a preservação do protocolo até o back.

Service Registry

InstanceRegistry + InstanceInfo implementam o catálogo dinâmico: quem existe, de que tipo (br/pt), onde (host:port) e se ainda está vivo (lastSeen). O registro é preenchido pelas próprias instâncias via REGISTER.

Heartbeat

HeartbeatClient (lado da instância) e HeartbeatServer (lado do gateway) implementam o sinal periódico de vida. Sem heartbeat dentro do timeout, removeDeadInstances() retira a réplica do mapa, ouse ja, ela deixa de ser escolhida no round-robin.

Load balancing (round-robin)

nextHealthy(serviceType) lista só instâncias vivas daquele componente e alterna com AtomicInteger. Com duas réplicas BR (br-1, br-2), os pedidos TIME br se distribuem entre elas enquanto ambas enviarem heartbeat.

3. Suporte a Múltiplos Protocolos

3.1 Protocolo UDP

1. ClientUdpServer escuta a porta 9090 e recebe um pacote UDP com o texto TIME br ou TIME pt.
 2. O TimeRequestHandler escolhe uma instância viva daquela zona.
 3. O UdpForwarder manda o mesmo texto, também em UDP, para a porta da instância.
 4. O InstanceUdpServer calcula o horário e responde OK ... (ou ERROR ...) em UDP.
 5. O gateway devolve essa resposta ao cliente, ainda em UDP.
- Porta: 9090
 - Classes: ClientUdpServer → UdpForwarder → InstanceUdpServer
 - Mensagem: TIME br|pt / resposta OK ...

3.2 Protocolo TCP

1. ClientTcpServer escuta a porta 9091. Cada cliente abre uma conexão e envia uma linha TIME br ou TIME pt.
 2. O TimeRequestHandler escolhe uma instância viva.
 3. O TcpForwarder abre um Socket até a instância, envia a linha e lê a resposta.
 4. O InstanceTcpServer valida a zona, monta o horário e responde com uma linha OK
 5. O gateway devolve essa linha ao cliente TCP.
- Porta: 9091
 - Classes: ClientTcpServer → TcpForwarder → InstanceTcpServer
 - Mensagem: uma linha de texto por conexão

3.3 Protocolo HTTP

1. ClientHttpServer escuta a porta 8080 e atende GET /time/br, GET /time/pt e GET /registry.
 2. Para /time/*, o TimeRequestHandler escolhe uma instância viva.
 3. O HttpForwarder faz um GET /time/{zona} HTTP até a porta da instância.
 4. Na instância, o InstanceTcpServer vê que o pedido começa com GET e responde em HTTP (não como linha TIME).
 5. O gateway devolve o corpo OK ... ao cliente (status 200; 502 se o forward falhar).
- Porta: 8080
 - Classes: ClientHttpServer → HttpForwarder → InstanceTcpServer (modo HTTP)
 - Mensagem: GET /time/br|pt

3.4 Protocolo gRPC

Não fiz :`(`

4. Ambiente de Testes na AWS

Os testes remotos rodaram em uma única instância EC2 na região us-east-2 (Ohio). O API Gateway e as quatro réplicas (br-1, br-2, pt-1, pt-2) executam nesse mesmo host e se comunicam por 127.0.0.1. O JMeter fica no PC (Windows, JMeter 5.6.3) e fala somente com o gateway, pelo Elastic IP.

4.1 Instâncias AWS EC2 e Configurações

Papel	Tipo instância	SO	vCPU / RAM	Observação
Gateway + workers BR/PT	t3.micro	Amazon Linux 2023 + Corretto 21 LTS	2 vCPU / 1 GiB	Processos: Main gateway, br-1, br-2, pt-1, pt-2
JMeter	PC local	Windows + JMeter 5.6.3	—	Aponta para o IP público do gateway

4.2 Endereços IP e Mapeamento de Rede

Componente	Endereço	Porta	Protocolo	Quem acessa
Gateway HTTP	3.146.239.232 (público)	8080	TCP	JMeter
Gateway UDP	3.146.239.232 (público)	9090	UDP	JMeter
Gateway TCP	3.146.239.232 (público)	9091	TCP	JMeter
SSH	3.146.239.232 (público)	22	TCP	PC local

Heartbeat	127.0.0.1	9000	TCP	Réplicas BR/PT
br-1	127.0.0.1	9101	TCP e UDP	Gateway
br-2	127.0.0.1	9102	TCP e UDP	Gateway
pt-1	127.0.0.1	9201	TCP e UDP	Gateway
pt-2	127.0.0.1	9202	TCP e UDP	Gateway

4.3 Outros Recursos Relevantes

Além da instância EC2, o ambiente usa estes recursos da AWS na região us-east-2:

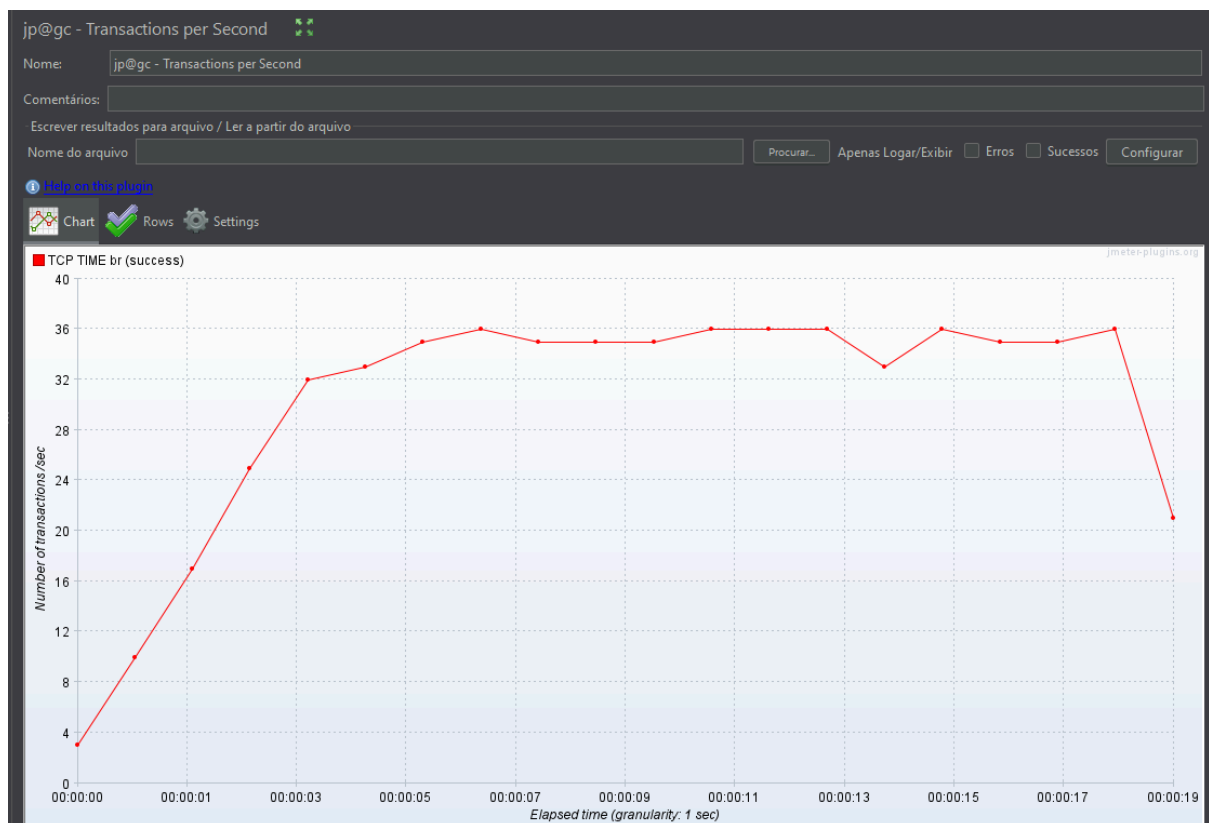
Recurso	Identificação	Função no laboratório
Elastic IP	3.146.239.232	Endereço público fixo do gateway para o JMeter e para o SSH
Security Group	launch-wizard-1 (sg-0e4e6ed231cc5eab6)	Libera 22/TCP, 8080/TCP, 9091/TCP e 9090/UDP. As portas 9000, 9101, 9102, 9201 e 9202 não são publicadas
Key pair	my-key.pem	Acesso SSH do usuário ec2-user
VPC padrão	endereço privado 172.31.39.59	Rede da instância na conta; não foi criada uma VPC separada

5. Resultados dos Testes de Carga

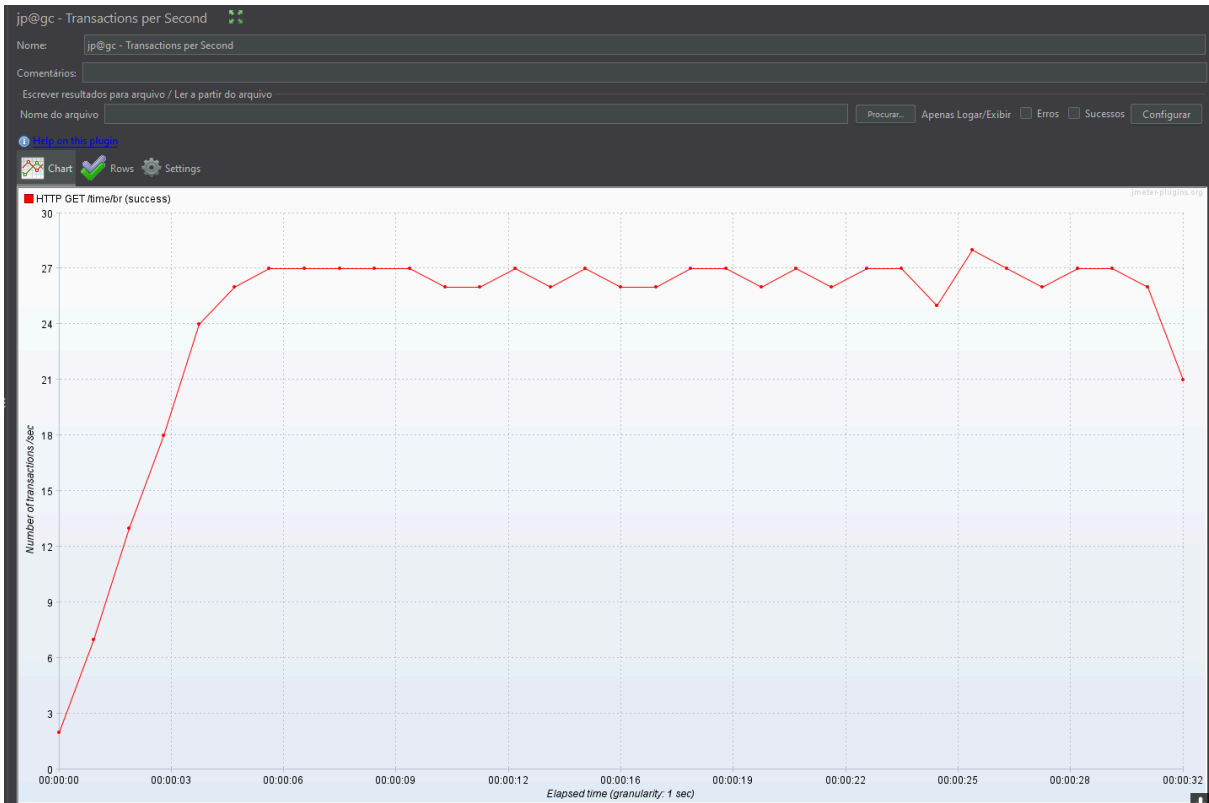
A varredura remota usou os planos isolados carga-tcp.jmx, carga-http.jmx e carga-udp.jmx, com o JMeter no PC e o gateway na EC2 (gatewayHost = 3.146.239.232), duas réplicas BR e duas PT. Cada ponto durou 20 s, com users em {1, 2, 3, 5, 8, 10, 15, 20, 30, 40, 50, 60, 80}.

5.1 Testes de Carga por Protocolo (UDP, TCP, HTTP)

TCP



HTTP



UDP

Relatório de Sumário

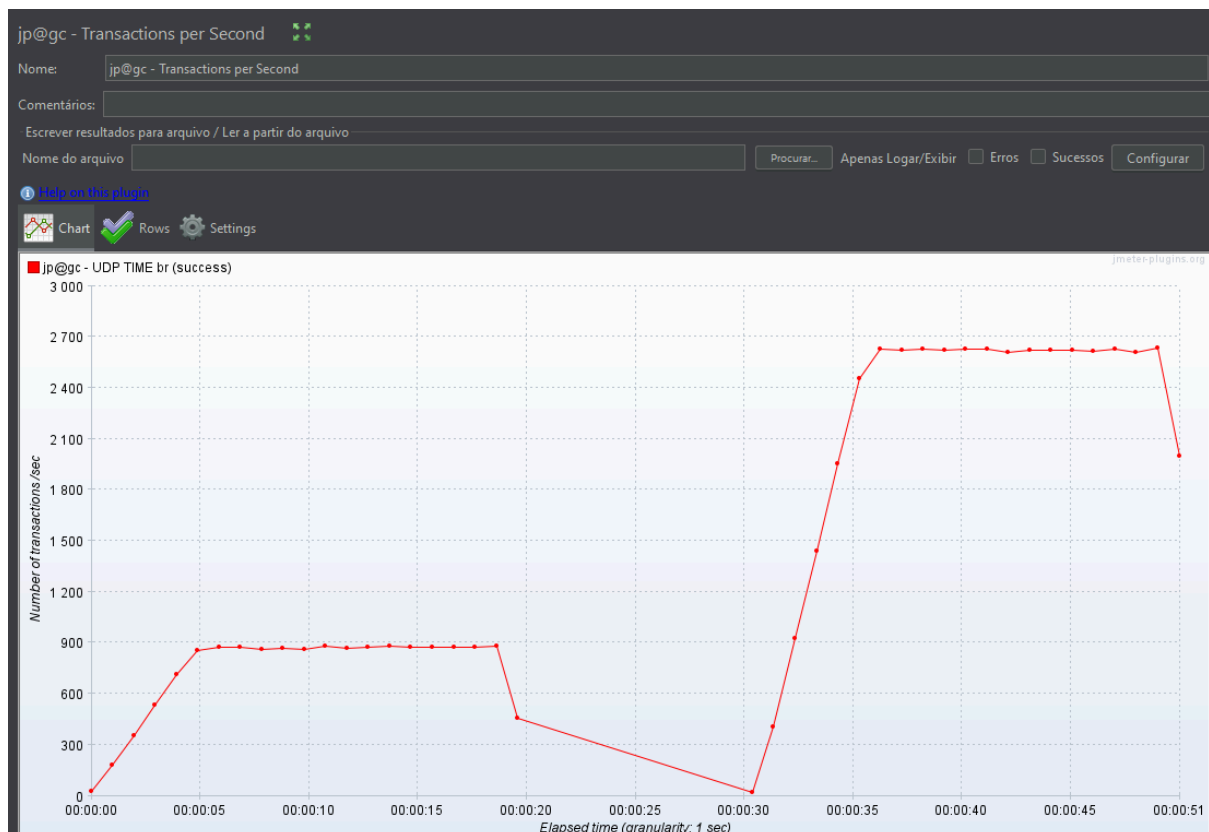
Nome: Sumário UDP

Comentários:

Escrever resultados para arquivo / Ler a partir do arquivo

Nome do arquivo: Procurar... Apenas Logar/Exibir ☐ Erros ☐ Sucessos Configurar

Rótulo	# Amostras	Média	Min.	Máx.	Desvio Padrão	% de Erro	Vazão	KB/s	Sent KB/sec	Média de Bytes
jp@gc - UDP Ti...	61378	114	104	170	6,02	0,00%	1194,7/sec	44,21	0,00	37,9
TOTAL	61378	114	104	170	6,02	0,00%	1194,7/sec	44,21	0,00	37,9



6. Discussão e Análise dos Resultados

6.1 Análise Comparativa dos Protocolos

Localmente, com o JMeter e a aplicação no mesmo computador, TCP, HTTP e UDP responderam sem erro. Na AWS, TCP e HTTP passaram a falhar em alguns momentos, sobretudo quando a carga aumentou. UDP, no mesmo ambiente, permaneceu estável.

A diferença mais plausível é a latência da rede até a instância em Ohio. No caminho remoto a vazão de TCP e HTTP satura cedo e parte dos pedidos expira. No localhost esse atraso não existe, e o mesmo código atende sem essas falhas. HTTP acompanha TCP porque o encaminhamento até a instância também usa conexão.

UDP não abre conexão por pedido, e por isso aguenta mais carga no mesmo enlace.

6.2 Considerações e Conclusões

A capacidade útil adotada para a avaliação é de 8 usuários simultâneos: acima de 5 e abaixo do ponto em que TCP e HTTP começam a falhar, com erro praticamente zero. Nesse patamar, desligar uma réplica aumenta a taxa de erro e subir outra faz o erro cair.

O gRPC não foi implementado. Gateway e Instâncias compartilham a mesma EC2, no Elastic IP 3.146.239.232.